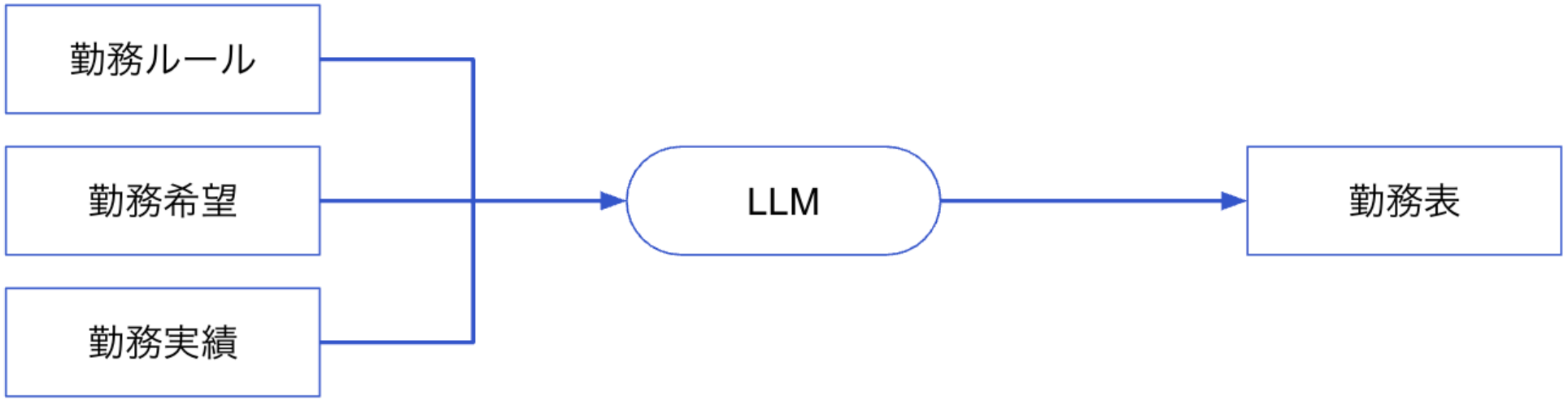
大規模言語モデルを用いた
自然言語入力による看護師勤務表生成手法の検討

太田 満久, 風間 正弘, 西林 孝
Ubie株式会社

ユビー

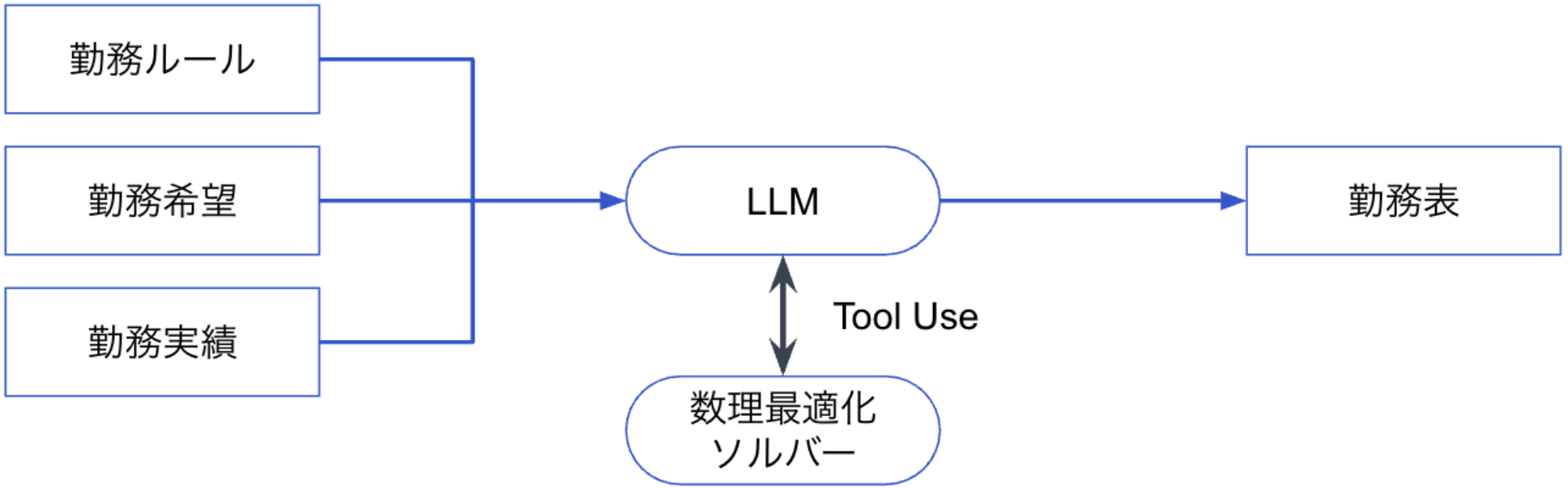
全体像

やりたいこと: LLMを用いて、自然言語で書かれた勤務ルールから勤務表を生成
手法: LLMが直接勤務ルールを出力する
課題: 現在のLLMでは、複雑なルールを守った勤務表を直接出力することはできない



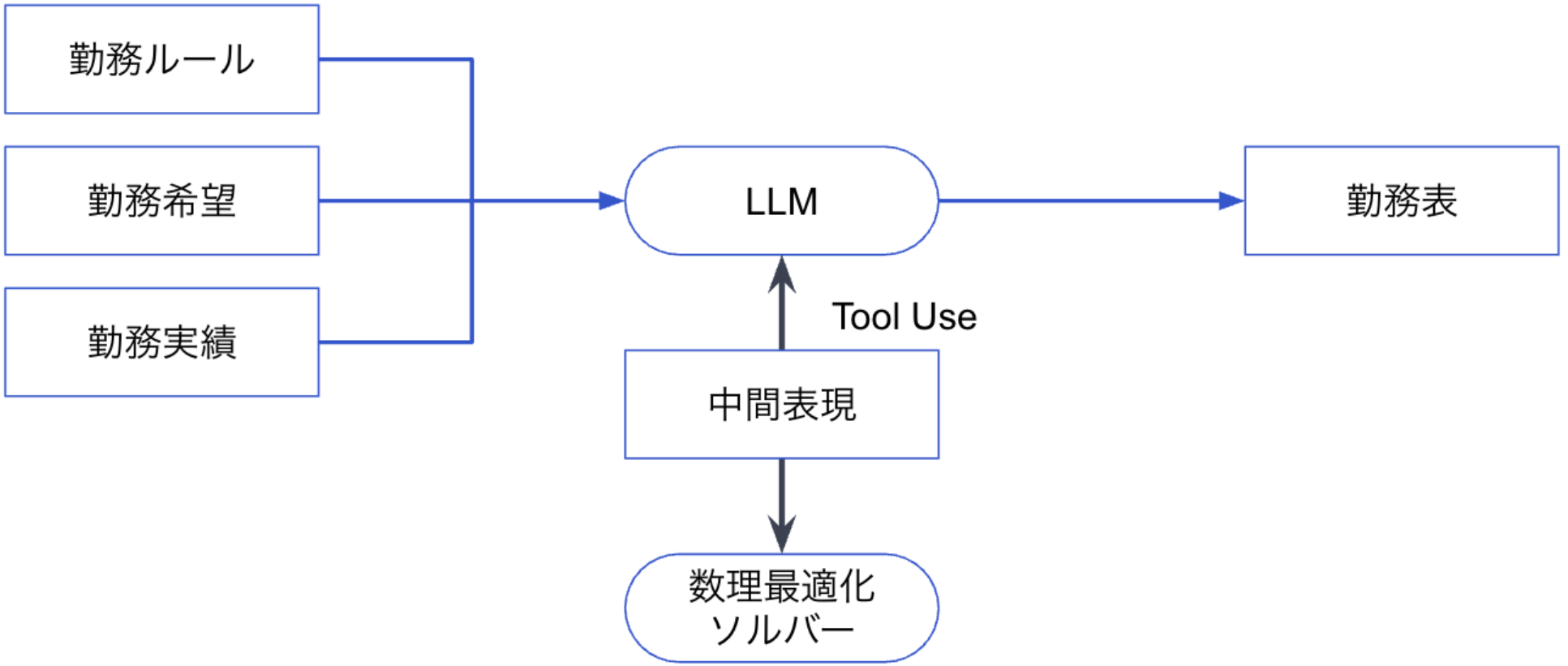
全体像

やりたいこと: LLMを用いて、自然言語で書かれた勤務ルールから勤務表を生成
手法: LLMが数理最適化ソルバーを直接利用し、勤務ルールを出力する（直接定式化）
課題: 数理最適化ソルバーを利用する際に抜け漏れ、間違いが発生



全体像

やりたいこと: LLMを用いて、自然言語で書かれた勤務ルールから勤務表を生成
提案手法: 中間表現と構造化出力を利用し、利用可能な機能を制限（分離アプローチ）
期待効果: 間違いや抜け漏れの抑制と、ルール遵守の確認が容易化



背景: 看護師の勤務表作成は複雑で煩雑

看護師の勤務表作成においては、法的な要件に加え、ガイドラインや病院・部署特有のローカルルールを守る必要がある

- **法的要件**: 労働基準法の遵守（労働時間、休日、休憩時間等）
- **勤務要件**: 夜勤回数の適正化や連続勤務日数・勤務間隔の確保
- **配置要件**: スキルレベルに応じた配置
- **個別対応**: 個人の希望や相性の考慮

勤務表の例

表 2.10　問題例 (Ikegami-3shift-DATA1) の最適解となる勤務表

	ナース番号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	休み	日勤	準夜勤	深夜勤	他勤務
		水	木	金	土	日	月	火	水	木	金	土	日	月	火	水	木	金	土	日	月	火	水	木	金	土	日	月	火	水	木	/	−	e	n	+
A	1	−	n	n	/	e	e	/	−	−	−	−	n	n	/	/	/	−	−	/	−	−	/	−	−	−	−	e	e	/	/	9	13	4	4	0
	2	n	/	e	e	/	−	−	−	/	/	−	−	+	/	−	n	n	/	e	e	/	−	e	n	n	/	/	−	−	−	9	10	5	5	1
	3	−	/	/	−	−	−	/	−	−	n	n	/	e	e	/	−	−	n	n	/	e	e	/	−	−	e	/	/	−	n	9	11	5	5	0
	4	/	−	−	−	/	−	−	n	n	/	/	/	−	−	−	−	e	e	/	+	−	n	n	/	/	/	−	−	e	e	9	12	4	4	1
	5	/	−	−	n	n	/	e	e	/	−	e	e	/	−	e	e	/	−	−	n	n	/	/	/	/	−	−	n	n	/	10	8	6	6	0
	6	e	e	/	/	−	n	n	/	e	e	/	−	−	n	n	/	/	/	−	−	/	−	−	e	e	n	n	/	/	−	10	8	6	6	0
	7	/	−	−	/	/	−	−	−	e	/	/	−	e	/	/	−	e	e	n	n	/	/	−	−	−	−	n	n	/	e	10	11	5	4	0
	8	n	n	/	/	−	e	e	/	−	+	−	−	−	−	/	−	−	−	e	/	/	−	−	/	/	/	e	e	/	−	10	12	5	2	1
	9	/	/	−	−	/	−	n	n	/	e	e	/	−	e	e	/	−	−	−	−	n	n	/	/	−	−	/	/	−	−	10	12	4	4	0
	10	+	/	−	−	−	−	/	/	−	n	n	/	/	−	−	+	/	−	−	e	/	−	−	e	e	/	−	−	e	/	9	13	4	2	2
	11	−	−	e	e	/	/	−	e	/	−	−	e	/	−	−	+	−	/	/	−	−	−	−	/	n	n	/	/	−	−	9	14	4	2	1
	12	−	e	/	−	n	n	/	/	/	−	−	−	−	/	−	+	−	/	−	−	−	/	−	−	−	e	/	/	/	−	10	15	2	2	1
	13	e	/	/	−	e	/	+	−	/	/	/	−	−	−	−	e	/	−	−	−	e	e	e	/	/	/	−	−	n	n	10	11	6	2	1
B	14	n	n	/	e	e	/	−	−	−	+	/	/	/	−	e	n	n	/	/	−	−	e	e	n	n	/	/	−	−	e	9	8	6	6	1
	15	−	−	−	/	n	n	/	/	/	−	e	e	e	/	−	−	e	/	−	e	e	/	−	−	−	/	−	n	n	/	9	11	6	4	0
	16	/	e	e	/	−	−	e	/	e	e	/	−	+	n	n	/	/	−	e	/	+	−	/	−	−	n	n	/	/	−	10	8	6	4	2
	17	e	/	−	−	/	−	−	e	n	n	/	/	+	−	−	e	/	n	n	/	/	−	−	e	e	e	/	/	−	−	9	10	6	4	1
	18	/	/	−	−	−	e	n	n	/	/	/	−	−	e	/	−	−	e	/	+	−	n	n	/	/	−	e	e	e	n	9	9	6	5	1
	19	−	−	−	/	−	−	+	n	n	/	/	/	−	−	−	+	−	e	/	−	e	e	/	/	/	/	−	−	−	−	9	14	3	2	2
	20	−	/	−	−	−	e	/	/	/	−	e	/	e	e	/	−	e	/	/	−	−	−	n	n	/	/	−	−	−	+	10	12	5	2	1
	21	/	e	e	e	/	−	−	/	−	−	−	e	n	n	/	/	/	−	−	−	−	/	−	−	−	−	/	/	/	+	10	13	4	2	1
	22	+	/	n	n	/	/	−	−	/	e	n	n	/	/	−	−	−	−	e	e	/	−	−	/	−	e	/	−	−	+	9	11	4	4	2
	23	/	−	−	−	/	−	−	−	/	/	−	−	/	−	e	e	n	n	/	/	−	−	−	−	e	/	e	e	/	+	9	13	5	2	1
	24	e	/	/	−	e	/	e	e	e	/	−	−	−	/	n	n	/	/	−	−	/	−	−	−	/	/	−	−	−	−	10	13	5	2	0
	25	−	−	n	n	/	/	/	−	−	/	−	n	n	/	/	/	−	−	−	n	n	/	e	e	/	−	−	−	e	e	9	11	4	6	0
日勤		8	8	11	11	8	11	9	9	7	7	9	10	9	10	10	8	10	10	10	11	9	11	13	9	9	7	9	10	10	10					
準夜勤		4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4					
深夜勤		3	3	3	3	3	3	3	4	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3					

出典: ナース・スケジューリング 問題把握とモデリング (近代科学社; 2018)

数理最適化による勤務表の自動作成

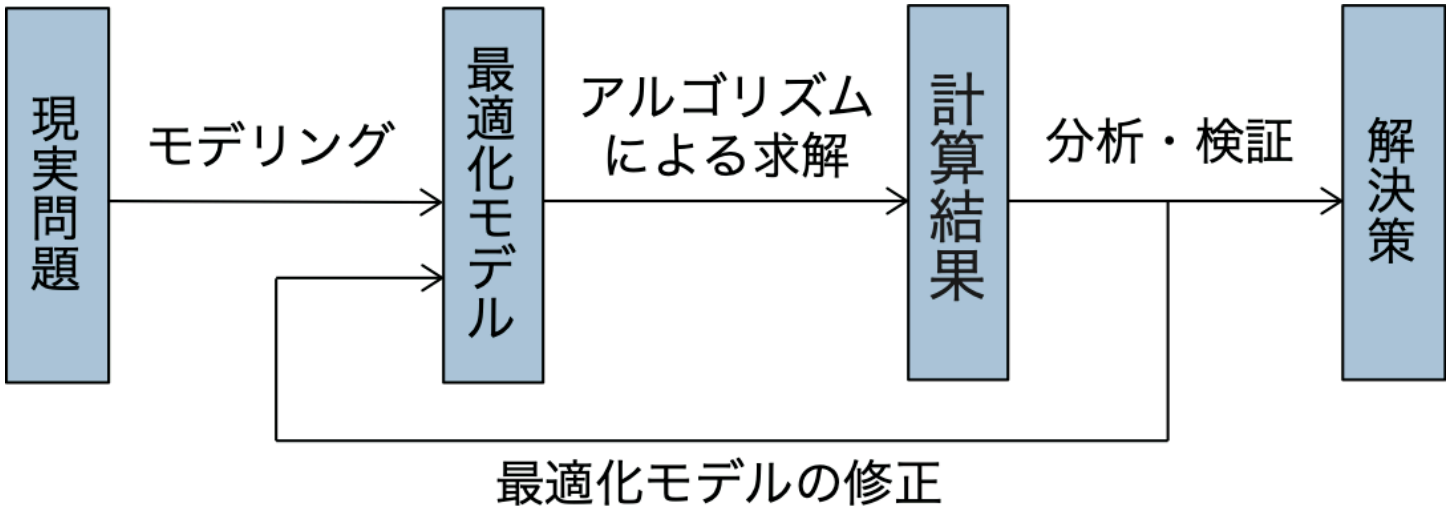
数理最適化分野において看護師の勤務表自動生成問題は「ナース・スケジューリング問題」と呼ばれ、多くの研究がなされている. ただし、広く実用化され普及しているとは言えない

- **数理最適化:**
  - 与えられた制約条件の下で目的関数を最大化・最小化する解を見つける手法
- **勤務表作成への応用:**
  - 目的関数: 公平性、希望休充足率、ルール逸脱数等
  - 制約条件: 法的要件、スキル、連続勤務上限、必要人数等
- **普及の上での課題:**
  - 既存のツールでは細かいルールに対応できないことが多い
  - 独自に仕組みを開発するにはコストがかかる

数理最適化の一般的なプロセス

数理最適化を用いた仕組みを独自に開発する場合、専門家とともに改善のループを回す必要がある

- 全てのルールをはじめから洗い出すのは困難
- モデリングや最適化モデルの修正には専門知識が不可欠

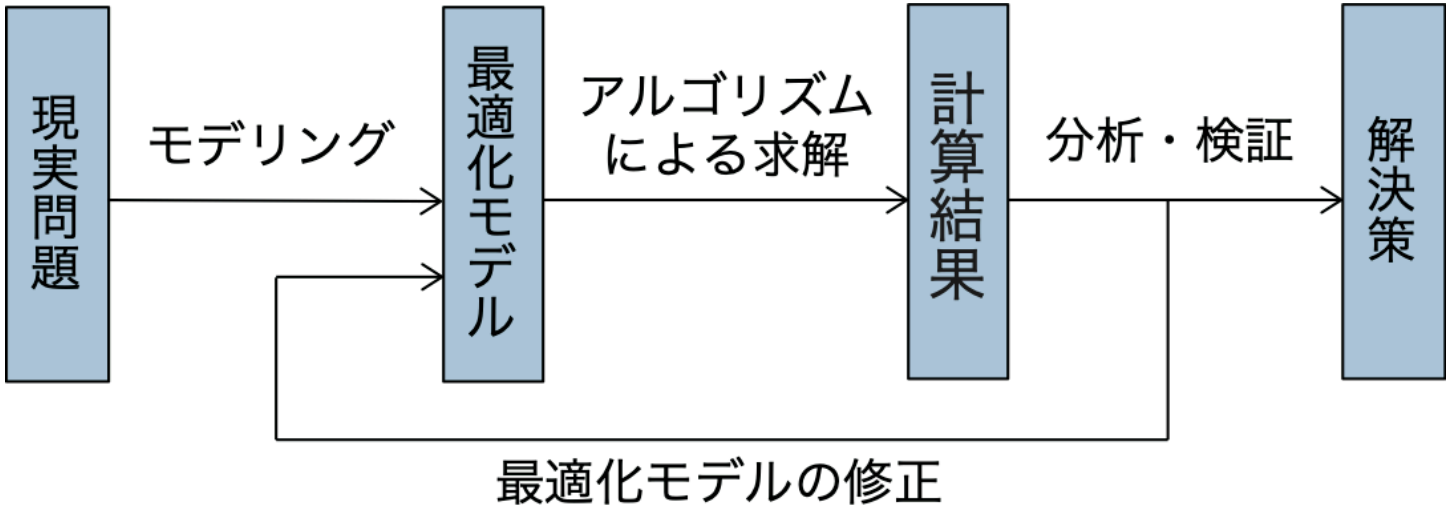


出典: 数理最適化による問題解決の実践的なアプローチ

数理最適化の一般的なプロセス

数理最適化を用いた仕組みを独自に開発する場合、専門家とともに改善のループを回す必要がある

- 全てのルールをはじめから洗い出すのは困難
- モデリングや最適化モデルの修正には専門知識が不可欠 → **LLM にやらせたい**

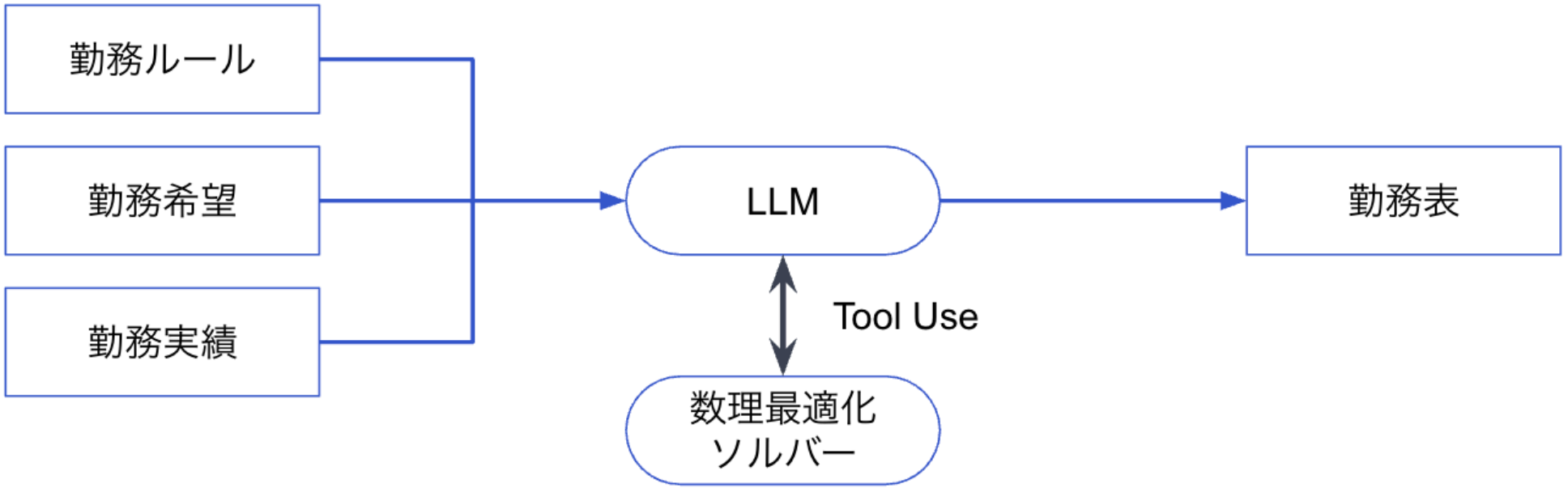


出典: 数理最適化による問題解決の実践的なアプローチ

検証1（直接定式化）: LLMで数理最適化のモデリングはできるか？

看護師の勤務表作成問題において、LLMが**直接**数理最適化ソルバーを利用して解を得ることができるか？

- 入力: 自然言語（Markdown形式）で書かれた勤務ルール
- 出力: 数理最適化ソルバーを利用するPythonコード



検証1（直接定式化）: LLMで数理最適化のモデリングはできるか？

補足: 簡単な問題については、最近のLLMだけで正確にモデリングできることがわかっている

巡回セールスマン問題をMIPで定式化して。出力はpulpを用いるPythonコードで。



```python
import pulp
import math
import random

def solve_tsp_mip(num_cities, points=None, solver_name=None):
    """
    巡回セールスマン問題をMIP（MTZ定式化）で解く関数。

    Args:
        num_cities (int): 都市の数。
        points (list of tuples, optional): 各都市の(x, y)座標のリスト。
                                           指定されない場合はランダムに生成される。
        solver_name (str, optional): 使用するソルバーの名前（"PULP_CBC_CMD", "GUROBI_CM
                                     Noneの場合はPuLPのデフォルトソルバーを使用。

    Returns:
        tuple: (status, total_distance, solution_path)
               status (str): 解の状態 ('Optimal', 'Not Solved', etc.)
               total_distance (float): 最適経路の総距離（解が見つかれば）
               solution_path (list of tuples): 最適経路（例: [(0, 1), (1, 2), (2, 0)
    """

    # 都市のインデックス
    cities = list(range(num_cities))

    # 都市の座標を生成（指定されていない場合）
    if points is None:
        points = [(random.randint(0, 100), random.randint(0, 100)) for _ in range(
    elif len(points) != num_cities:
        raise ValueError("pointsリストの長さがnum_citiesと一致しません。")

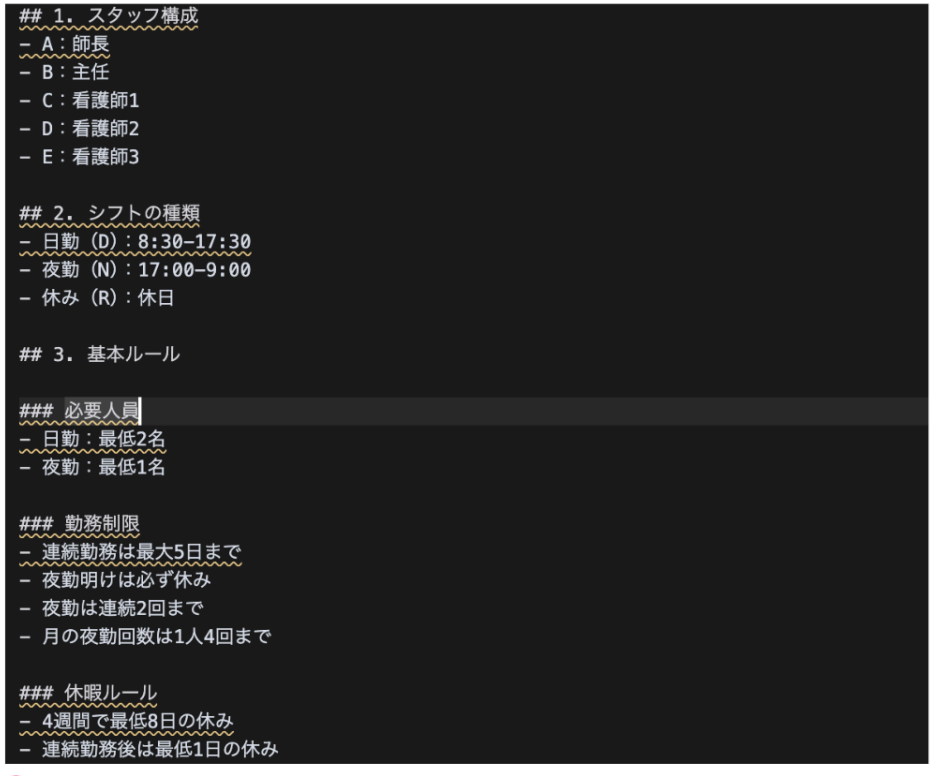
    # 距離行列の計算（ユークリッド距離）
    def distance(p1, p2):
        return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

    dist = [[0] * num_cities for _ in range(num_cities)]
```

Google Gemini 2.5 Pro (preview) の回答結果

検証1（直接定式化）：LLMで数理最適化のモデリングはできるか？

現実的な問題においても、正確にモデリングできるか？



```
## 1. スタッフ構成
- A：師長
- B：主任
- C：看護師1
- D：看護師2
- E：看護師3

## 2. シフトの種類
- 日勤（D）：8:30-17:30
- 夜勤（N）：17:00-9:00
- 休み（R）：休日

## 3. 基本ルール

### 必要人員
- 日勤：最低2名
- 夜勤：最低1名

### 勤務制限
- 連続勤務は最大5日まで
- 夜勤明けは必ず休み
- 夜勤は連続2回まで
- 月の夜勤回数は1人4回まで

### 休暇ルール
- 4週間で最低8日の休み
- 連続勤務後は最低1日の休み
```





```
import pulp
import math
import random

def solve_tsp_mip(num_cities, points=None, solver_name=None):
    """
    巡回セールスマン問題をMIP（MTZ定式化）で解く関数。

    Args:
        num_cities (int): 都市の数。
        points (list of tuples, optional): 各都市の(x, y)座標のリスト。
                                           指定されない場合はランダムに生成される。
        solver_name (str, optional): 使用するソルバーの名前 ("PULP_CBC_CMD", "GUROBI_CM
                                     Noneの場合はPuLPのデフォルトソルバーを使用。

    Returns:
        tuple: (status, total_distance, solution_path)
               status (str): 解の状態 ('Optimal', 'Not Solved', etc.)
               total_distance (float): 最適経路の総距離（解が見つかれば）
               solution_path (list of tuples): 最適経路 (例: [(0, 1), (1, 2), (2, 0)
    """

    # 都市のインデックス
    cities = list(range(num_cities))

    # 都市の座標を生成（指定されていない場合）
    if points is None:
        points = [(random.randint(0, 100), random.randint(0, 100)) for _ in range(n
    elif len(points) != num_cities:
        raise ValueError("pointsリストの長さがnum_citiesと一致しません。")

    # 距離行列の計算（ユークリッド距離）
    def distance(p1, p2):
        return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

    dist = [[0] * num_cities for _ in range(num_cities)]
```

検証1（直接定式化）: LLMで数理最適化のモデリングはできるか？

- **言語モデル:** OpenAI GPT-4o, o1, o3-mini
- **利用データ** 公開データ（Ikegami-3shift-DATA1）及び独自収集の実際の勤務ルール
- **入力形式:** 人間が理解しやすい形式（Markdown）でルールを記述したもの

  ルールの記述例

```
## 4. 勤務条件
### 看護師A
- 日勤: 月5回まで, 夜勤: 月4回まで ...
### 全体ルール
- 夜勤翌日の日勤は不可, 必要最低人数: 2名
```

- **出力形式:** 数理最適化ソルバーを利用するPythonコード

検証1（直接定式化）: LLMで数理最適化のモデリングはできるか？

結果

1. 現実問題に多い複雑な問題については、正しくモデリングできないことが多い
2. 正しくモデリングできているかの確認に専門知識が必要で労力を要する

```
...
prob.addConstraint(x[7][1]['/'] == 1, name="Nurse7Day1IsOff")

# (h) この他、間隔制約(7日に1回は/がある等)を実装するなら追加
#     例: 任意の7連区間の和が>=1 など
#     ここでは未実装


# 6. 求解の実行
solution_status = prob.solve(pulp.PULP_CBC_CMD(msg=0))
...
```

o1 の出力例. 複雑な部分を省略して出力してしまう

検証1（直接定式化）：LLMで数理最適化のモデリングはできるか？

結果

1. 現実問題に多い複雑な問題については、正しくモデリングできないことが多い
2. 正しくモデリングできているかの確認に専門知識が必要で労力を要する

```
# (f) 特別制限: ナース1とナース9は e・n で同時不可
for d in range(1, num_days+1):
    prob.addConstraint(
        x[1][d]['e'] + x[9][d]['e'] <= 1,
        name=f"NoSimul_e_day{d}")
    prob.addConstraint(
        x[1][d]['n'] + x[9][d]['n'] <= 1,
        name=f"NoSimul_n_day{d}")
```

上記の条件は「ナース1と9が同時に同じシフトに入らない」ことを表現しているか？

検証2（分離アプローチ）: LLMは適切な中間表現を出力できるか？

- **言語モデル:** OpenAI GPT-4o, o1, o3-mini
- **利用データ** 公開データ（Ikegami-3shift-DATA1）及び独自収集の実際の勤務ルール
- **入力形式:** 人間が理解しやすい形式（Markdown）でルールを記述したもの

  ルールの記述例

```
## 4. 勤務条件
### 看護師A
- 日勤: 月5回まで, 夜勤: 月4回まで ...
### 全体ルール
- 夜勤翌日の日勤は不可, 必要最低人数: 2名
```

- **出力形式:** 制約条件を表すデータクラス（**構造化出力**）

制約条件を表すデータクラスの例

LLMは制約条件に必要なパラメータのみ（以下の例では pattern, staffs）を出力すれば良く、制約の実現方法に関心を払う必要がない

※ コードそのものを出力するのではなく、構造化出力の仕組みを用いる

```
class ForbiddenShiftPatternPolicy(Policy):
    """禁止シフトパターンに関する制約を表現するクラス

    staffs について、 pattern の並び順のシフトは禁止される
    """
    pattern: List[Union[ShiftType, ShiftTypeGroup]] = Field(..., description="禁止するシフトパターン")
    staffs: Optional[List[Union[Staff, StaffGroup]]] = Field(None, description="対象となるスタッフのリスト。
    指定しない場合は全てのスタッフに適用される")
```

製薬の内容は非専門家が理解可能

適切な `__str__` を実装しておけば、非専門家でも容易に解釈可能

```
class ForbiddenShiftPatternPolicy(Policy):
    """禁止シフトパターンに関する制約を表現するクラス

    staffs について、 pattern の並び順のシフトは禁止される
    """
    pattern: List[Union[ShiftType, ShiftTypeGroup]] = Field(..., description="禁止するシフトパターン")
    staffs: Optional[List[Union[Staff, StaffGroup]]] = Field(None, description="対象となるスタッフのリスト。
    指定しない場合は全てのスタッフに適用される")

    def __str__(self) -> str:
        policy = f"{self.name}: "

        if self.staffs is not None:
            staff_names = "".join(f"'{staff.name}'" for staff in self.staffs)
            policy += f"スタッフ {staff_names} について、"

        policy += f"次のパータンを禁止: "
        policy += " > ".join(f"'{shift.name}'" for shift in self.pattern)

        return policy
```

製薬の内容は非専門家が理解可能

適切な `__str__` を実装しておけば、非専門家でも容易に解釈可能

```
- 4週間で8日以上休日: シフト 年 は、期間全体で 8回以上
- 年末年始休暇: シフト 年 は、メンバーが1人以上
- ゴールデンウィーク休暇: シフト 年 は、メンバーが1人以上
- 年間休日総数111日（4月1日起算）: シフト 年 は、期間全体で 111回以上111回以下
- 年5日の有給休暇取得: シフト 年 は、期間全体で 5回以上
- Aチーム夜勤体制: シフト × は、スタッフ Aチーム のメンバーが1人以上
- Bチーム夜勤体制: シフト × は、スタッフ Bチーム のメンバーが1人以上
- Cチーム夜勤体制: シフト × は、スタッフ Cチーム のメンバーが1人以上
- 夜勤リーダー必須: シフト × は、スキル 青 のあるメンバーが1人以上
- Aチーム日勤最低人数: シフト 日 は、スタッフ Aチーム のメンバーが2人以上
- Bチーム日勤最低人数: シフト 日 は、スタッフ Bチーム のメンバーが2人以上
- Cチーム早番必須: シフト 早 は、スタッフ Cチーム のメンバーが1人以上
- Cチーム遅番必須: シフト 遅 は、スタッフ Cチーム のメンバーが1人以上
...
```

参考（再掲）直接定式化の場合の出力

直接定式化の場合は、LLMの出力が正しいかどうか、専門知識がなければ判断できない。また、専門家でも正しいかどうかの確認に労力を要する

```
# (f) 特別制限: ナース1とナース9は e・n で同時不可
for d in range(1, num_days+1):
    prob.addConstraint(
        x[1][d]['e'] + x[9][d]['e'] <= 1,
        name=f"NoSimul_e_day{d}")
    prob.addConstraint(
        x[1][d]['n'] + x[9][d]['n'] <= 1,
        name=f"NoSimul_n_day{d}")
```

検証に用いた制約条件を表すデータクラス（一部抜粋）

以下のデータクラスの組み合わせでルールを表現する

制約クラス名	説明
ShiftAssignmentPolicy	特定のシフトへの割り当て可否
FollowableShiftPolicy	あるシフトの次に割り当て可能なシフト
PrecedingShiftPolicy	あるシフトの前に必要なシフト
ForbiddenShiftPatternPolicy	禁止される勤務パターン
ShiftEligibilityPolicy	特定のスキルを持つ看護師のみ割り当て可能
ShiftCountPolicy	一定期間内の勤務回数制限

補足: ポリシーから数理最適化の制約条件への対応

ポリシーごとに数理最適化ソルバーの解釈できる制約条件への変換する関数を実装しておく

```
    def _apply_forbidden_shift_pattern_policy(self, policy: ForbiddenShiftPatternPolicy):
        slacks = []
        staffs = []
        if policy.staffs is not None:
            for staff_or_group in policy.staffs:
                staffs.extend(get_staffs(staff_or_group))
        else:
            staffs = self.staffs.values()

        len_pattern = len(policy.pattern)
        for staff in staffs:
            # for date in self.opt_dates:
            for date in self.lookback_dates:
                if date.date + datetime.timedelta(days=len_pattern) > self.opt_end_date:
                    continue
                slack = self._get_slack(policy.is_mondatory, f"policy_{policy.name}__staff_{staff.name}__date_{date.date}")
                slacks.append(slack)
                self.model.addCons(
                    quicksum(
                        self._x[staff.name, date.date + datetime.timedelta(days=i), shift_type.name]
                        for i, shift_type_or_group in enumerate(policy.pattern)
                        for shift_type in get_shift_types(shift_type_or_group)
                    ) - slack <= len_pattern - 1
                )

        return slacks
```

検証2（分離アプローチ）: 結果

入力と一対一に対応するような制約条件は、GPT-4o、o1, o3-mini ともに正しく出力できる

入力ルール

```
### 日勤帯の必要人数
- 全体: 13-15名
- Aチーム: 6-8名
- Bチーム: 5-7名
- B準ベテラン: 2-4名
- B準新人: 2-4名
```

LLMの出力（データクラスの文字列表現）

```
- Aチームの日勤帯の必要人数: シフト - は、スタッフ Aチーム のメンバーが6人以上8人以下
- Bチームの日勤帯の必要人数: シフト - は、スタッフ Bチーム のメンバーが5人以上7人以下
- B準ベテランの日勤帯の必要人数: シフト - は、スタッフ B準ベテラン のメンバーが2人以上4人以下
- B準新人の日勤帯の必要人数: シフト - は、スタッフ B準新人 のメンバーが2人以上4人以下
```

GPT-4oの出力結果

検証2（分離アプローチ）：結果

入力と一対一に対応しない制約条件など、複雑な場合は正しく出力できないことがある

入力ルール

```
### 特別制限
- ナース1とナース9は準夜勤・深夜勤で同時勤務不可
```

LLMの出力（GPT-o4、 誤り）

```
- ナース1とナース9の準夜勤・深夜勤同時勤務不可: スタッフ 'ナース1"ナース9' について、次のパータンを禁止: 'e' > 'n'
```

LLMの出力（o1、正解）

```
## シフトグループ
-EorN: 準夜勤または深夜勤 (e, n)

## ルール
- ナース1とナース9は準夜勤または深夜勤を同時不可: シフト EorN は、スタッフ Nurse1 or Nurse9 のメンバーが0人以上1人以下
```

検証2（分離アプローチ）：結果

o1は、条件をスキップすることがある

入力ルール

```
### 連続勤務制限
- 休み: 1-5日連続
- 日勤: 1-4日連続
- 準夜勤: 1-3日連続
- 深夜勤: 必ず2日連続
…
```

LLMの出力（データクラスの文字列表現）

```
-禁止パターン: 深夜勤(n)直後に(+): 次のパータンを禁止: 'n' > '+'
-禁止パターン: 深夜勤(n)直後に(-): 次のパータンを禁止: 'n' > '-'
-禁止パターン: 深夜勤(n)直後に(e): 次のパータンを禁止: 'n' > 'e'
-禁止パターン: 準夜勤(e)直後に(+): 次のパータンを禁止: 'e' > '+'
-禁止パターン: 準夜勤(e)直後に(-): 次のパータンを禁止: 'e' > '-'
-禁止パターン: 準夜勤(e)直後に(n): 次のパータンを禁止: 'e' > 'n'
-禁止パターン: 休み(/)直後に(-): 次のパータンを禁止: '/' > '-'
-禁止パターン: 休み(/)直後に(+): 次のパータンを禁止: '/' > '+'
-禁止パターン: 休み(/)直前に(n): 次のパータンを禁止: 'n' > '/'
```

連続勤務制限が欠落している

検証2（分離アプローチ）: 結論

- シンプルなルールについては、正しい出力ができる
- 複雑なルールの場合、解釈を間違えてしまうことがある（特に GPT-4o）
- 直接定式化と同様、ルールをスキップしてしまうことがある（o1 を含む）

まとめ

1. 直接定式化と分離アプローチで、LLMによる看護師勤務表作成に取り組んだ
2. 直接定式化と分離アプローチともに、ルールの取りこぼしや定式化のミスが発生しうるため、実用にはさらなる改善が必要
    - 現段階の実装では、全てのルールを一度に生成するようになっている点が大きいと考えられる
    - プロンプトのチューニングが不足している可能性もある
3. 直接定式化と異なり、分離アプローチでは、非専門家であっても正しく定式化できたか容易に確認ができる
    - 専門家を介さずに改善のループを回せる（はず）